

Twin AI Technical Specification

PROJECT	MODULE	STATUS
Cosmiclly RTC Marketplace	AI Digital Clone (Text & Voice)	Architecture Draft (Phase 2)

1. System Overview

The Twin AI allows Advisors to activate a digital clone that interacts with Clients when the human Advisor is offline or unavailable. The AI must accurately represent the Advisor's tone, knowledge, and pricing structure.

The AI Stack:

- **Knowledge Base (RAG):** Supabase `pgvector` + OpenAI `text-embedding-3-small`.
- **Reasoning / Text Generation:** OpenAI `gpt-4o` (or `gpt-4o-mini` for cost efficiency).
- **Voice Cloning:** Cartesia.
- **Real-Time Voice Infrastructure:** Vapi.ai (Web SDK) handles the low-latency WebRTC connection, orchestrating OpenAI and ElevenLabs.

2. Database Schema Additions (Supabase)

```
To support the Twin AI, we must extend the existing database schema.
-- 1. Enable pgvector extension for the knowledge base
create extension if not exists vector;

-- 2. Extend advisor_details table
alter table public.advisor_details
add column twin_enabled boolean default false,
add column twin_text_rate_per_msg integer default 0, -- Separate pricing
for AI
add column twin_voice_rate_per_min integer default 0,
add column system_prompt text, -- Custom instructions (e.g., "Act as a
compassionate tarot reader...")
add column cartesia_voice_id varchar(255),
add column vapi_agent_id varchar(255);
```

```
-- 3. Create Knowledge Base table for RAG
create table public.knowledge_base_documents (
  id uuid primary key default gen_random_uuid(),
  advisor_id uuid references public.profiles(id) on delete cascade,
  content text not null, -- The actual text chunk
  embedding vector(1536), -- OpenAI embedding size
  source_filename varchar(255),
  created_at timestamp with time zone default timezone('utc'::text,
now())
);

-- 4. Create an index to speed up vector similarity search
create index on public.knowledge_base_documents
using hnsw (embedding vector_cosine_ops);
```

3. Core Workflows

Flow A: Knowledge Ingestion (Training the Twin)

Goal: Allow the AI to answer questions using the Advisor's unique background.

1. **Upload:** Advisor uploads a PDF, text file, or links past readings in the React UI.
2. **Storage:** File is temporarily saved to Supabase Storage (`training_docs` bucket).
3. **Processing (Edge Function - `ingest-knowledge`):**
 - Downloads the file.
 - Splits the text into smaller chunks (e.g., 500 tokens each).
 - Calls OpenAI Embeddings API to convert chunks into vectors.
 - Inserts rows into `knowledge_base_documents`.

Flow B: Voice Cloning (Cartesia)

Goal: Generate the `cartesia_voice_id`.

1. **Recording:** In the `AdvisorPrivateProfile` UI, the Advisor reads a 60-second provided script into their microphone.
2. **Submission (Edge Function - `clone-voice`):**
 - Receives the audio blob.
 - Sends the audio to Cartesia's Voice Cloning API endpoint.
 - Receives a generated voice ID and saves it to `advisor_details.cartesia_voice_id`.
3. **Vapi Sync:** The function then calls Vapi's API to create or update the `vapi_agent_id`, linking the new Cartesia voice ID to their Vapi profile.

Flow C: Text Mode (Offline Chatbot)

Goal: AI responds to text messages in the Supabase Realtime chat.

1. **Trigger:** Client sends a message in `Chat.tsx`. It saves to the `messages` table.
2. **Database Webhook:** A Supabase Database Webhook detects the insert. If `advisor_id.twin_enabled` is true AND the advisor's status is `offline`:
 - Trigger Edge Function `handle-ai-chat`.
3. **Retrieval (RAG):** The Edge Function embeds the Client's message and queries `knowledge_base_documents` for the top 3 most relevant chunks of the Advisor's knowledge.
4. **Generation:** Edge Function calls OpenAI with:
 - *System Prompt:* The advisor's custom instructions.
 - *Context:* The 3 retrieved knowledge chunks.
 - *History:* The last 5 messages in the thread.
5. **Response:** The Edge Function inserts the AI's reply back into the `messages` table (with an `is_ai_generated: true` flag to visually distinguish it in the UI).

Flow D: Voice Mode (Real-Time AI Call)

Goal: Client speaks with the AI over a low-latency voice call. (Note: We bypass LiveKit entirely for AI calls to reduce latency and complexity. Vapi handles the WebRTC transport natively).

1. **Initiation:** Client clicks "Call Twin AI".
2. **Billing Check:** System verifies Client has funds (Auth & Capture).
3. **Connection:** `VoiceCall.tsx` initializes the **Vapi Web SDK** using the Advisor's `vapi_agent_id`.
4. **Conversation:** Vapi natively handles STT (Speech-to-Text) -> OpenAI -> Cartesia (TTS) -> Client Audio.
5. **Termination & Billing:** When the call ends, Vapi sends a webhook to Supabase (`vapi-webhook` Edge Function) containing the exact call duration to finalize the Stripe charge.

4. Required Supabase Edge Functions to Build

1. `ingest-knowledge`: Parses text/PDFs, generates embeddings, saves to `pgvector`.
2. `clone-voice`: Proxies audio to Cartesia and updates Vapi configurations.
3. `handle-ai-chat`: Runs the RAG loop and replies in text threads.
4. `vapi-webhook`: Receives end-of-call events from Vapi to process billing and save the call transcript to the `sessions` table.

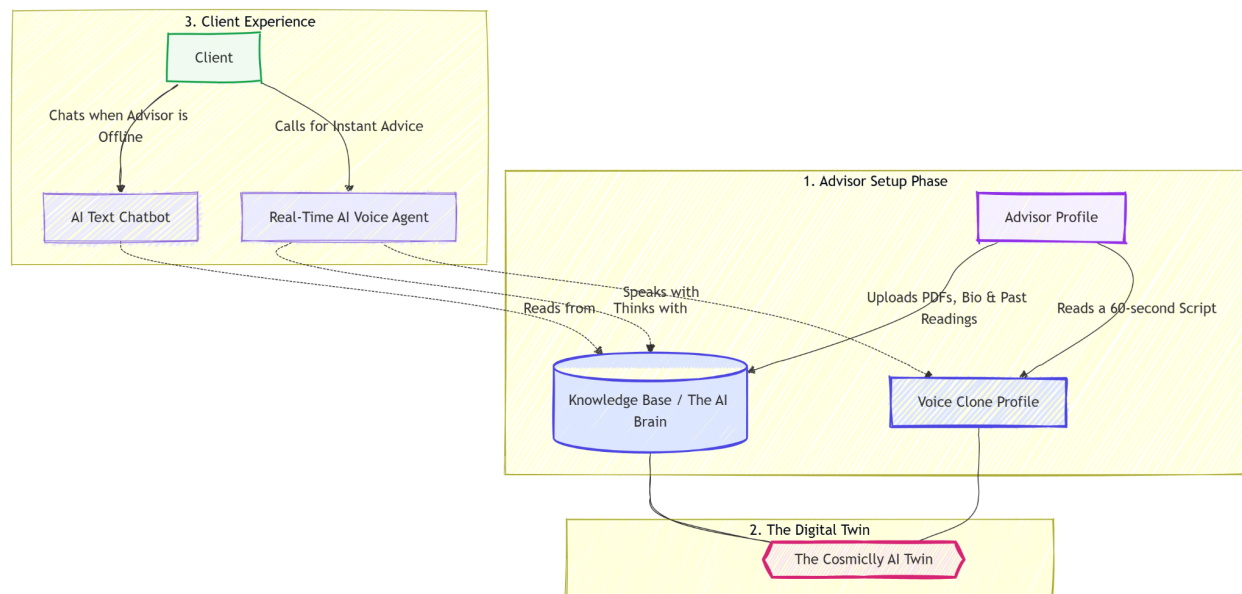
5. UI/UX Requirements

- **Advisor Portal:** Needs a "Twin Setup" wizard (Record Voice, Upload Docs, Write System Prompt, Toggle ON/OFF).
 - **Client View:** Clear visual indicators so the Client knows they are interacting with the "Twin AI" and not the human (to prevent deception).
 - **Pricing Display:** Show distinct rates (e.g., Human: \$3.00/min | Twin AI: \$0.50/min).
-

6. Architecture Diagrams

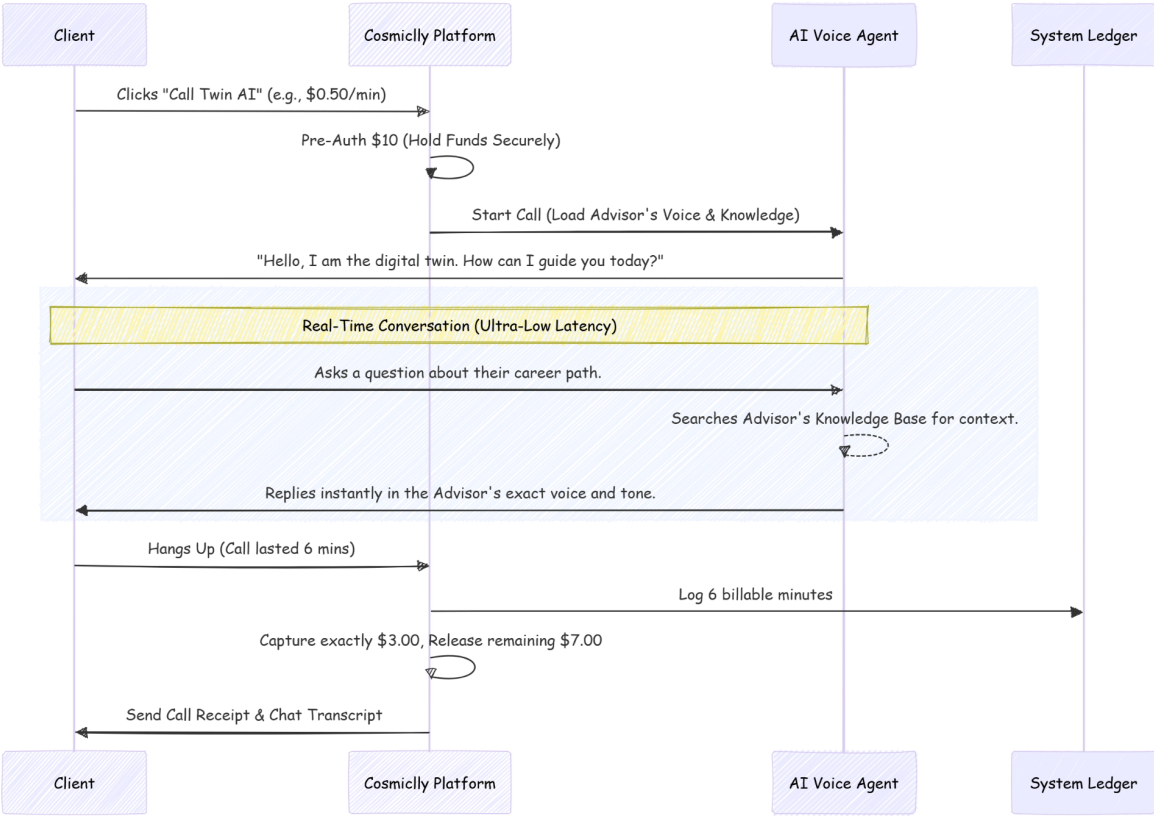
6.1 Ecosystem Overview

This diagram illustrates the separation between the Advisor's setup process (training the AI) and the Client's real-time experience.



6.2 Voice Call Flow (Real-Time Experience)

This sequence illustrates the low-latency interaction between the Client and the Twin AI, including the Stripe Auth & Capture billing steps.



DOCUMENT REVISION HISTORY

Version Number	Date	Revisions	Author
1.0	February 19, 2026	-Initial creation	Ivan Pamonag